Al fin(ito) y más allá del bearer token

Un billete al portador llamado token

Seguridad de APIs · de Basic al DPoP

2026 · charla técnica · 28 diapositivas

Un token es un billete al portador: quien lo tiene, entra

- posesión = autorización
- "bearer" viene del derecho mercantil
- no identifica al portador
- RFC 6750 (oct. 2012)

GUION DEL PONENTE

Un bearer token es un artefacto de seguridad cuya posesión equivale a autorización. La palabra 'bearer' viene del derecho mercantil: un instrumento al portador da derecho a quien lo tiene en la mano, sin acreditar identidad. Si perdés un billete, quien lo levanta lo gasta. Si se filtra un token, quien lo tiene lo usa. No es que el token esté mal hecho: está hecho así a propósito

Fuente: RFC 6750, "Bearer Token Usage", octubre 2012.

La norma lo admitió primero: no puedo saber a quién le emití esto

> *«La especificación no provee ningún método para que el servidor de recursos se asegure de que un access token presentado por un cliente le fue emitido realmente a ese cliente.»*
>
> **RFC 6749, §10.3 — octubre 2012**

GUION DEL PONENTE

Esto no lo descubrimos después: está en la propia norma. RFC 6749, sección 10.3, dice, textualmente, que la especificación no provee ningún método para que el servidor de recursos se asegure de que un access token presentado por un cliente le fue emitido realmente a ese cliente. En criollo: desde 2012 el estándar dice que un servidor no puede distinguir entre el cliente legítimo y quien le robó el token. Por eso el bearer no falló: cumplió exactamente lo que prometió

Fuente: RFC 6749 §10.3, octubre 2012.

Alrededor del token hay un sistema entero



GUION DEL PONENTE

Antes de seguir con los tokens, ubiquémonos: una API no es solo el token. Alrededor hay un gateway que enruta y aplica cuotas; un IAM que administra identidades; la MFA que protege el login; el rate limiting que frena la fuerza bruta; un WAF que filtra ataques; API keys para integradores; OAuth y OIDC como marco; el JWT como formato; los scopes y modelos como RBAC o ABAC para decidir permisos; un secret manager para guardar claves; mTLS y DPoP para atar el token a su portador; observabilidad y auditoría para ver qué pasa; y Zero Trust como filosofía. El token es una pieza del sistema: si lo miramos aislado, no entendemos nada

Fuente: OWASP API Security Top 10 (2023).

No todas las APIs son REST: cada estilo tiene su propio riesgo

REST	GraphQL	gRPC	SOAP	WebSocket	Webhooks
recursos	*consultas*	*binario*	*XML*	*canal*	*push*
BOLA / IDOR	query depth / sobre-exposición	WAF no inspecciona	XXE	auth solo al inicio	SSRF / replay

GUION DEL PONENTE

Antes de seguir, aclaremos una cosa: no todas las APIs son REST. REST trabaja con recursos y trae el riesgo de BOLA y de IDOR. GraphQL expone un solo endpoint y trae consultas profundas y sobre-exposición de campos. gRPC es binario y rápido, pero los proxies y WAF no lo inspeccionan fácil. SOAP usa XML y arrastra XXE. WebSocket abre un canal persistente y suele autenticar solo al inicio. Y los webhooks y los eventos son push: si no los firmás y validás la URL, tenés SSRF y replay. La receta de REST no se copia y pega: cada estilo necesita su propio modelo

Fuente: OWASP API Security Top 10 (2023).

Primero mandábamos la contraseña; después, una cookie

- Basic manda usuario:contraseña en Base64 por request
- Digest mejora el hash, no el modelo
- la cookie guarda estado en el servidor

GUION DEL PONENTE

Todo empieza con HTTP sin estado. La primera solución fue mandar usuario y contraseña en cada petición: Basic, dentro del marco de autenticación de HTTP estandarizado en el RFC 2617, de junio de 1999. Basic codifica en Base64 —que no es cifrado— y Digest manda un hash con un nonce. Pero las credenciales siguen viajando en cada request. El segundo intento fue no mandar credenciales, sino un identificador de sesión: la cookie, de Netscape (1996), que de hecho es anterior a la estandarización de Digest, y que recién se estandarizó en el RFC 6265, en abril de 2011. El estado queda en el servidor: seguro para la web tradicional, incómodo para APIs que tienen que delegar

Fuente: RFC 2617 (1999) / RFC 7616 (2015); RFC 6265 (2011).

Lo firmado era más seguro; lo simple ganó igual

- SAML 2.0 (OASIS, 2005) y WS-Security (2004) firmaban aserciones XML
- OAuth 1.0 (RFC 5849, 2010) firmaba cada request con HMAC-SHA1
- más seguro, más pesado

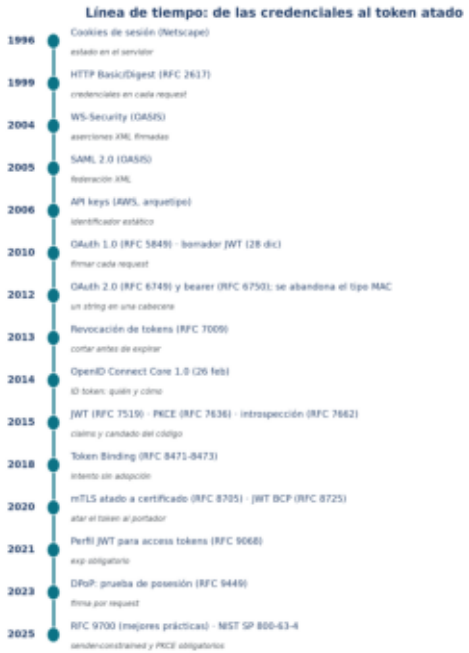
GUION DEL PONENTE

En el mundo enterprise, SAML 2.0, estándar OASIS de marzo de 2005, y WS-Security de 2004 usaban aserciones XML firmadas: eran, en rigor, más seguros que un bearer. El problema no era la seguridad, era el peso. Después llegó OAuth 1.0, especificación comunitaria de diciembre de 2007 y RFC 5849 en abril de 2010: no mandaba ningún token copiable, firmaba cada request con HMAC-SHA1. Robar una petición no servía. Pero canonicalizar parámetros y compartir secretos entre cada par de servicios era una pesadilla, sobre todo en móviles

Fuente: SAML 2.0 (2005); WS-Security (2004); RFC 5849 (2010).

OAuth 2.0 nació con dos tipos de token; el mundo eligió uno

- RFC 6749 (oct. 2012) definía bearer y mac
- el tipo mac firmaba cada request
- el borrador MAC (feb. 2012) se abandonó
- ganó la simplicidad



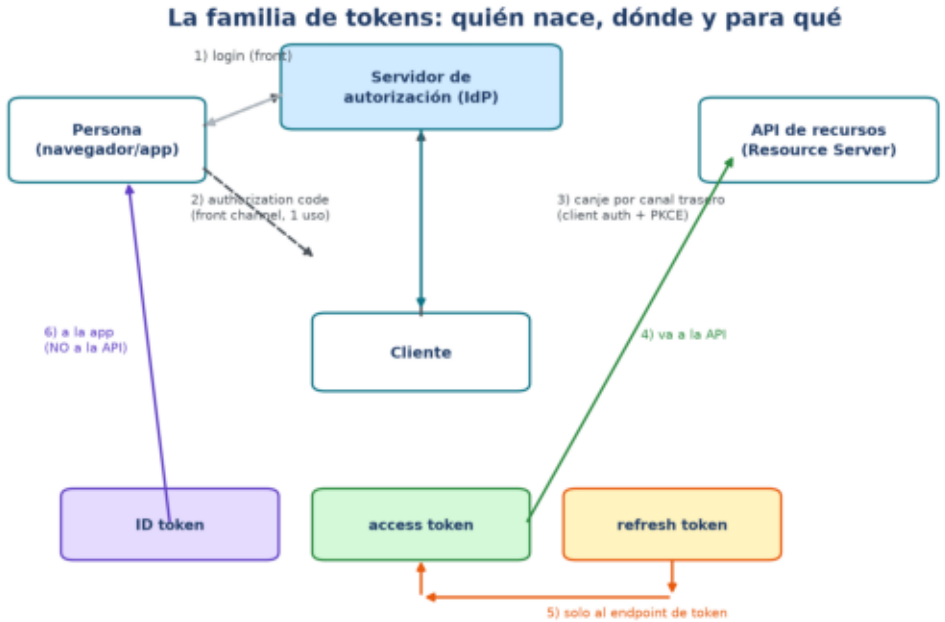
GUION DEL PONENTE

Este es el dato histórico que casi nadie cuenta: OAuth 2.0 no nació siendo solo bearer. RFC 6749, de octubre de 2012, definía dos tipos: bearer y mac. El tipo mac firmaba componentes de cada request, igual que OAuth 1.0 o que DPoP después. Pero ese borrador nunca llegó a RFC: se abandonó. El ecosistema eligió la simplicidad, y con él llegó RFC 6750. La firma por request era técnicamente superior; el costo de implementación decidió la historia

Fuente: RFC 6749 §7.1 (2012); borrador MAC, febrero 2012.

No hay 'un token': hay una familia con roles distintos

- authorization code (un solo uso) → access token (corto) → refresh token (largo, rotación) + ID token (identifica, no autoriza)



GUION DEL PONENTE

Un error común es hablar de 'el token' como si fuera uno solo. En un flujo moderno nacen hasta cuatro artefactos distintos. El authorization code es un cupón de un solo uso que viaja por el navegador. El access token es el que se presenta ante la API: vida corta, alcance limitado. El refresh token es de vida larga y solo sirve contra el endpoint de token, nunca contra la API. Y el ID token, de OpenID Connect, dice quién es la persona y cómo se autenticó: no sirve para acceder a recursos. Confundir estos roles es el origen de la mayoría de las fallas

Fuente: RFC 6749 §4.1 / §5 (2012); OIDC Core, 26 de febrero de 2014.

Cada token tiene su destinatario y su vida

- código (front channel, un uso)
- access (la API, corto)
- refresh (el endpoint de token, largo, rota)
- ID (la app, identifica)

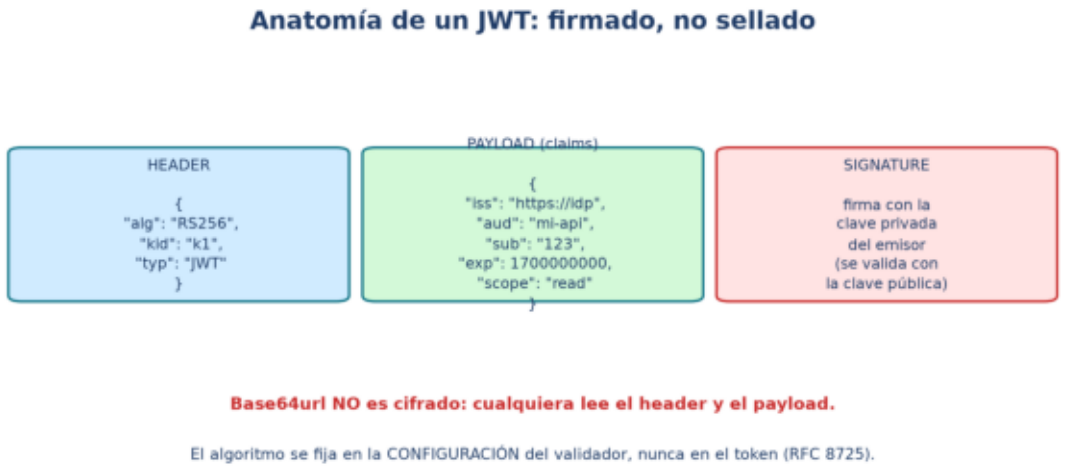
GUION DEL PONENTE

Pongámoslo en una tabla. El código de autorización es de un solo uso y viaja por el navegador. El access token viaja en cada request a la API y debe vivir minutos u horas. El refresh token vive semanas o meses, pero solo se presenta al endpoint de token. El ID token va a la aplicación cliente, no a la API. Regla para escribir en grande: el refresh token jamás viaja a la API de recursos, y el access token jamás se usa para refrescar. Son mundos distintos: recursos contra emisión

Fuente: RFC 6749 §§1.5, 4.1, 5, 6 (2012); RFC 7009 (2013).

Está firmado, no sellado: el payload lo lee cualquiera

- header.payload.signature en base64url
- base64 no es cifrado
- el algoritmo NO se toma del token (RFC 8725)



GUION DEL PONENTE

Un JWT son tres partes separadas por puntos, cada una en base64url. Cuidado con el error clásico: base64 no es cifrado, cualquiera lee el payload. Es como una credencial plastificada: firmada, no sellada; si necesitás confidencialidad, va JWE. El header lleva el algoritmo, y ahí nace una familia entera de ataques: si el receptor decide cómo verificar según lo que dice el token, un atacante elige el algoritmo. La regla, de RFC 8725 y de OWASP: la política de verificación sale de la configuración, nunca del mensaje

Fuente: RFC 7519 (2015); RFC 8725 (feb. 2020).

Un JWT bien validado responde cuatro preguntas

- ¿quién lo emitió? (iss)
- ¿es para mí? (aud)
- ¿está vigente? (exp/nbf)
- ¿tiene permiso? (scope)
- amr/acr dicen cómo se autenticó

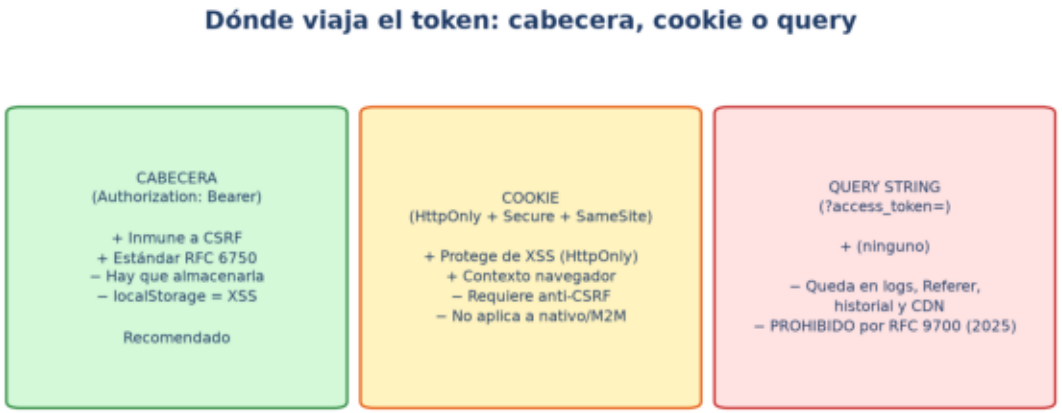
GUION DEL PONENTE

Los claims son el contenido. iss dice quién lo emitió; hay que validarlo contra emisores confiables. aud dice para quién es: es el claim que responde a la pregunta de la norma, porque si validás aud estrictamente, un token robado emitido para otra app no te sirve. exp y nbf dicen si está vigente. scope dice qué puede hacer. Y amr y acr, de RFC 8176, dicen cómo se autenticó: son el puente entre 'pasó la MFA' y 'el token lo sabe'

Fuente: RFC 7519 (2015); RFC 8176 (2017); RFC 9068 (2021).

Trece años tardamos en prohibir el token en la URL

- RFC 6750 permitía cabecera, cuerpo y query
- RFC 9700 (enero 2025) prohíbe tokens en URIs
- cabecera vs cookie: XSS vs CSRF



GUION DEL PONENTE

En 2012, RFC 6750 aceptaba tres transportes: cabecera, cuerpo y parámetro de query. Trece años después, RFC 9700, las mejores prácticas de enero de 2025, prohíbe transportar access tokens en URIs. ¿Por qué el cambio? Porque trece años de logs, de cabeceras Referer, de historiales y de CDN nos enseñaron dónde terminan los tokens que van en una URL. La cabecera no la manda el navegador sola: es inmune a CSRF, pero hay que guardarla bien. La cookie protege de XSS con HttpOnly, pero exige mitigar CSRF

Fuente: RFC 6750 §2 (2012); RFC 9700 (enero 2025).

La MFA protege un instante; el token vive 30 días

- el flujo real es password → MFA → emisión → token (30 días) → API
- la API no sabe si hubo MFA salvo que se lo digan los claims



GUION DEL PONENTE

Esta es la idea que justifica el título. La autenticación multifactor no ocurre en la API: ocurre en el momento de la emisión. Después de la MFA, todo lo que la API recibe es un string. El flujo real es: contraseña, MFA en el servidor de autenticación, emisión de tokens y, al final, un access token. La API no sabe, y no puede saber, si hubo MFA; solo lo sabe si el emisor puso amr y acr en el token y el validador los revisa. La MFA protege la generación del artefacto, no su posesión posterior

Fuente: NIST SP 800-63-4 (jul. 2025); OIDC Core (2014).

Passkeys son el único factor que sobrevive al phishing

- NIST AAL1-AAL3
- WebAuthn/passkeys (W3C, 2019) resisten phishing
- TOTP se phishea en tiempo real
- SMS es el más débil
- step-up con acr/amr

GUION DEL PONENTE

NIST, en su guía de identidad vigente de julio de 2025, dice algo clave: no todos los autenticadores dan la misma garantía, y el nivel debe ligarse al riesgo. AAL2 exige dos factores de tipos distintos; AAL3 exige prueba criptográfica de posesión de una clave en hardware. WebAuthn y las passkeys, recomendación W3C de 2019, resisten el phishing por diseño: el desafío está atado al origen. Un OTP se puede phishear en tiempo real, y el SMS es el más débil. Y el step-up, con acr y amr, permite pedir más garantía solo para la operación sensible

Fuente: NIST SP 800-63-4 (jul. 2025); WebAuthn (2019); RFC 8176 (2017).

La MFA cubre el login, no lo que pasa después

- fatiga de push (MITRE T1621)
- phishing AiTM roba la sesión ya autenticada
- infostealers y pass-the-cookie roban el artefacto

GUION DEL PONENTE

Hay tres familias de ataque que la MFA no cubre, porque atacan después de la emisión. Uno: fatiga de MFA, o push bombing —MITRE lo cataloga como T1621—: el atacante ya tiene la contraseña y dispara pushes hasta que la víctima aprueba. Dos: phishing adversary-in-the-middle: ya no roba la contraseña, intercepta la sesión completa que resultó de la MFA. Tres: infostealers, malware que extrae cookies y tokens del navegador y los reutiliza en otro equipo; se llama pass-the-cookie. En los tres, la MFA hizo su trabajo; el problema es que el artefacto se reutiliza

Fuente: MITRE ATT&CK T1621; T1539; casos públicos 2022.

A una máquina no le podés pedir un segundo factor

- el invocador es microservicio, CI o IoT
- no hay usuario
- la pregunta es qué workload habla y con qué credencial de vida corta
- mecanismos: client credentials, private_key_jwt, mTLS, DPoP

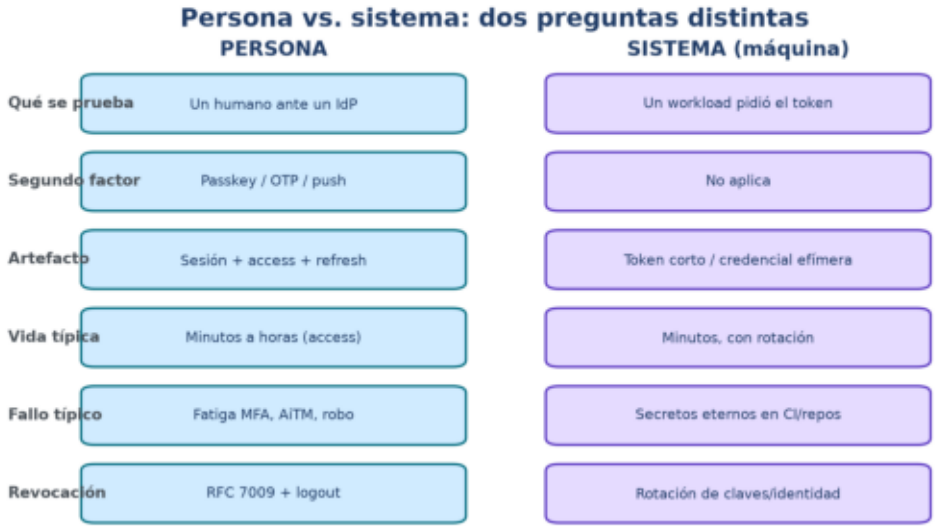
GUION DEL PONENTE

Ahora el otro lado, donde está la mayoría del tráfico real. Cuando la API la invoca un microservicio, un job de CI o un dispositivo, la pregunta '¿quién es el usuario?' no tiene respuesta, porque no hay usuario. Y no hay MFA aplicable: no hay segundo factor que mostrar. La pregunta correcta es: ¿qué workload es este, con qué credencial de ciclo de vida corto y bajo qué contexto habla? El equivalente funcional del MFA para un sistema no es un segundo factor puntual: es una identidad de workload fuerte, verificable y de vida corta

Fuente: RFC 6749 §4.4 (2012); RFC 7523 (2015); RFC 8705 (2020).

Dos preguntas distintas, no una sola mal aplicada

- persona: ¿quién pasó la MFA?
- sistema: ¿qué credencial de vida corta me habla?
- el fallo típico de cada lado



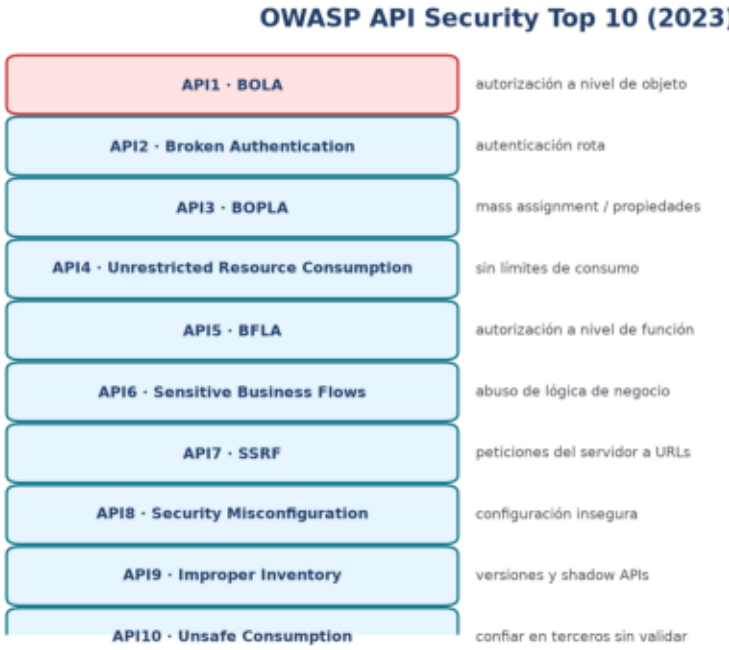
GUION DEL PONENTE

Pongamos las dos columnas juntas. Para la persona probamos que un humano se autenticó ante un proveedor de identidad; el segundo factor es una passkey, un OTP o un push; el artefacto es una sesión más un access token corto; y el fallo típico es fatiga, AiTM o robo de sesión. Para el sistema probamos que un workload concreto pidió el token; no hay segundo factor; el artefacto es un token corto o una credencial efímera; y el fallo típico son secretos eternos en repos y CI. Aplicar el marco de uno al otro es el error más caro de la disciplina

Fuente: NIST SP 800-63-4 (2025); GitGuardian Secrets Sprawl 2026.

El token es una puerta; los riesgos viven en todo el edificio

- BOLA (Api1)
- Broken Authentication (Api2)
- BOPLA (Api3)
- consumo y versión (Api4/9)
- negocio y configuración



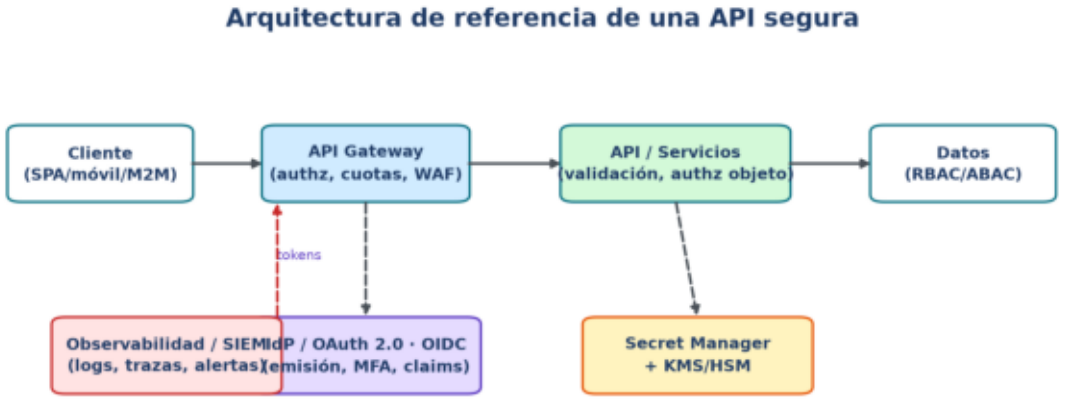
GUION DEL PONENTE

OWASP nos da el mapa de lo que puede salir mal. Arriba de todo, Api1: BOLA, autorización a nivel de objeto. Sigue Api2, autenticación rota; Api3, exposición de propiedades, también llamado mass assignment; Api4, consumo sin límites; Api5, autorización a nivel de función; Api6, flujos de negocio sensibles; Api7, SSRF; Api8, mala configuración; Api9, inventario impropio; y Api10, consumo inseguro de terceros. Ninguno de estos es 'el token': son el sistema alrededor del token. Por eso el token perfecto no alcanza

Fuente: OWASP API Security Top 10, edición 2023.

El token es una puerta; el edificio tiene más habitaciones

- autorización a nivel de objeto
- validación e inyección
- gateway y rate limiting
- inventario y versiones
- cadena de suministro



Cada capa agrega un control. El token es solo una de ellas.

GUION DEL PONENTE

El token resuelve la entrada, pero el edificio tiene más habitaciones. Primero, la autorización a nivel de objeto: que puedas ver el pedido 5678 y no solo el 1234. Segundo, la validación de entrada: inyección, SSRF, deserialización, mass assignment. Tercero, el gateway y el control de tráfico: cuotas, rate limiting y gestión de bots. Cuarto, el inventario: versiones viejas y APIs fantasma que siguen vivas. Quinto, la cadena de suministro: secretos en CI y dependencias de terceros. Y sexto, privacidad y cumplimiento: qué datos personales tratás y qué te exige la norma, incluida la de Uruguay. El token perfecto no tapa ninguna de estas habitaciones

Fuente: OWASP API Top 10 (2023); NIST; AGESIC / CERTuy.

El token funcionó exactamente como estaba diseñado

- Facebook "View As" (2018): token emitido al usuario equivocado
- Okta/Cloudflare (2023): sesión robada y no revocada
- Codecov (2021): tokens de CI filtrados

GUION DEL PONENTE

Los casos que siguen son divulgaciones públicas y responsables; no vamos a señalar a nadie, vamos a observar el modelo. Facebook, 2018: la función 'View As' generó un access token del usuario consultado, no del espectador, y lo dejó embebido en el HTML; se resetearon unos 90 millones de cuentas. Okta y Cloudflare, 2023: en el ataque al sistema de soporte de Okta, un session token de un ticket se extrajo y siguió sirviendo, porque no se rotó ni se revocó a tiempo. (En enero de 2022 Okta ya había sufrido otro incidente; en aquel caso Cloudflare no fue impactado gracias a sus llaves de hardware.) Codecov, 2021: un script modificado exportó las variables de entorno del CI y con ellas los tokens y claves. En los tres, el token no falló: funcionó como fue diseñado

Fuente: Meta (2018); Cloudflare (2023); Codecov (2021).

Después de aceptarlo, todavía queda trabajo

- Uber (2022): la MFA no sobrevive al robo de sesión
- BOLA: token válido, objeto ajeno (~40 % de ataques a APIs)
- JWT mal validado: alg:none, confusión HS/RS, kid

GUION DEL PONENTE

Uber, 2022: credenciales de un contratista compradas en un foro, un SMS que simulaba al área de IT y la aprobación del push; la MFA protegió el instante, pero la sesión se reutilizó. BOLA: el atacante presenta su propio token, válido y con MFA si quieren, y cambia el identificador del objeto en la URL; funciona, y se estima que ronda el 40 % de los ataques a APIs. Y las clases de JWT mal validado: alg:none, confusión RS256 a HS256, inyección de kid. Ninguna es culpa del string: son fallas de emisión, autorización o verificación

Fuente: UpGuard/NYT (2022); OWASP 2023; RFC 8725 (2020).

Si la posesión no basta, exijamos prueba de posesión

- mTLS + token atado al certificado (RFC 8705, 2020)
- DPoP firma por request con jti/htm/htu (RFC 9449, 2023)
- RFC 9700 (2025) recomienda sender-constrained



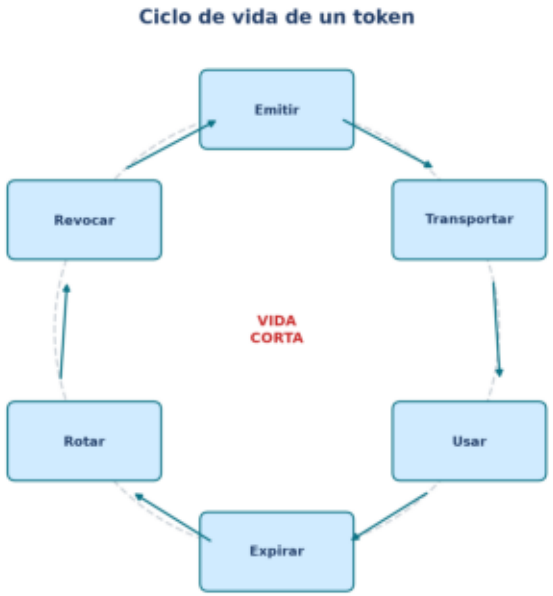
GUION DEL PONENTE

Si el problema es que cualquiera que tenga el token lo usa, la solución es exigir algo más que la posesión. Hay dos caminos estandarizados. mTLS más token atado al certificado, RFC 8705 de 2020: el token queda ligado al certificado del cliente, así que robarlo no alcanza sin el par de claves. Y DPoP, RFC 9449 de octubre de 2023: el cliente firma cada request con su clave privada, con un jti único y los datos del método y la URL. No necesita PKI. Y esto ya no es opcional: RFC 9700, de 2025, recomienda tokens sender-constrained donde sea posible

Fuente: RFC 8705 (2020); RFC 9449 (2023); RFC 9700 (2025).

La rotación convierte el refresh en un artefacto de un solo uso

- el refresh rotado delata el reuso
- RFC 7009 define la revocación
- introspección (RFC 7662) para tokens opacos
- relación entre vida del JWT y revocación



GUION DEL PONENTE

El refresh token es el que renueva sin volver a pedir permiso, y su mejor defensa es la rotación: cada vez que se usa, se emite uno nuevo y el viejo se invalida. Si un atacante lo usa, el legítimo lo delata en el siguiente intento. RFC 6749 lo sugería en 2012; RFC 9700 lo vuelve obligatorio para clientes públicos. Y para lo que hay que cortar antes de tiempo está la revocación, RFC 7009 de 2013; y para tokens opacos, la introspección, RFC 7662. Regla de oro: no podés tener a la vez un JWT de larga vida y la capacidad de revocarlo. Elegí uno

Fuente: RFC 6749 §10.4 (2012); RFC 7009 (2013); RFC 9700 (2025).

El token se acaba; el riesgo no

- vida corta y aud estricto
- rotación y revocación
- mínimo privilegio
- detección de anomalías
- secret manager para claves de máquina

GUION DEL PONENTE

Juntemos las prácticas. Vida corta y audiencia estricta para el access token; rotación y revocación para el refresh; scopes mínimos; y observabilidad: ¿este token se usa desde dos países a la vez? Para las máquinas, la regla es secreto en un secret manager, rotación automática y nada de claves eternas en repos ni en variables de entorno. Y recordá el dato que le da urgencia a todo esto: la mayoría de los secretos filtrados no son contraseñas de personas, son credenciales de sistemas, y más del 60 % de los secretos válidos que se detectaron en 2022 seguían activos años después

Fuente: RFC 9700 (2025); RFC 7009 (2013); GitGuardian 2026.

Ocho preguntas antes de dar por segura una API

- ¿el algoritmo viene de la config?
- ¿validás iss/aud/exp?
- ¿la vida es corta?
- ¿hay rotación y revocación?
- ¿autorizás por objeto?

GUION DEL PONENTE

Ocho preguntas para llevarse. Uno: ¿el algoritmo de verificación sale de la configuración y no del token? Dos: ¿validás issuer, audiencia y expiración con tolerancia de reloj corta? Tres: ¿el access token vive lo que debe vivir? Cuatro: ¿hay rotación y revocación de refresh? Cinco: ¿la autorización a nivel de objeto está resuelta, o solo la autenticación? Seis: ¿el token viaja en cabecera y nunca en la URL? Siete: ¿las identidades de máquina usan credenciales de vida corta y rotación? Ocho: ¿qué queda en los logs? Si alguna respuesta te incomoda, ahí está tu próximo trabajo

Fuente: RFC 8725 (2020); RFC 9700 (2025); OWASP API Top 10 (2023).

Un dominio, un mensaje: lo que hay que llevarse

- identidad (¿quién o qué?)
- autorización (¿sobre qué objeto?)
- entrada (¿confío? no)
- tráfico (¿límites?)
- observabilidad (¿lo veo?)

GUION DEL PONENTE

Resumamos por dominio, una frase cada uno. Identidad: preguntate siempre si el invocador es una persona o un sistema, porque la respuesta cambia todo. Autorización: la pregunta no es si el token es válido, sino si esta identidad puede tocar este objeto. Entrada: la respuesta correcta es 'no confío'; validá todo. Tráfico: poné límites, porque el abuso y la denegación de servicio son baratos para el atacante. Observabilidad: si no lo ves, no lo puedes defender ni revocar. Y cumplimiento: mirá qué te exige la norma; en Uruguay, la Ley 18.331, el Decreto 66/025 y el reporte a CERTuy. Seis dominios, seis preguntas, y ninguna la responde el token solo

Fuente: OWASP API Top 10 (2023); NIST SP 800-63-4 (2025); RFC 9700 (2025).

Al fin(ito) y más allá: el token se acaba, el riesgo no

- (a) el token es finito: emitir, rotar, revocar
- (b) la MFA protege la emisión, no la posesión
- (c) autenticar es la puerta, autorizar es la cerradura

GUION DEL PONENTE

Cerremos con tres ideas. Primero: el bearer token es finito: se emite, se roba, se reutiliza y debe morir; medí su vida, rotalo, revocalo, porque la ausencia de expiración ya es una vulnerabilidad. Segundo: la MFA protege el momento de la emisión, no los treinta días siguientes; diseñá para el robo de artefactos, no solo para el robo de contraseñas; y si tu invocador es un sistema, no busques un MFA, buscá identidad de workload de vida corta. Tercero: autenticar es la puerta; autorizar es la cerradura; un token perfecto no impide BOLA. Al fin(ito) y más allá: la seguridad nunca estuvo en el string, estuvo siempre en el sistema. Gracias

Fuente: bibliografía consolidada al final.

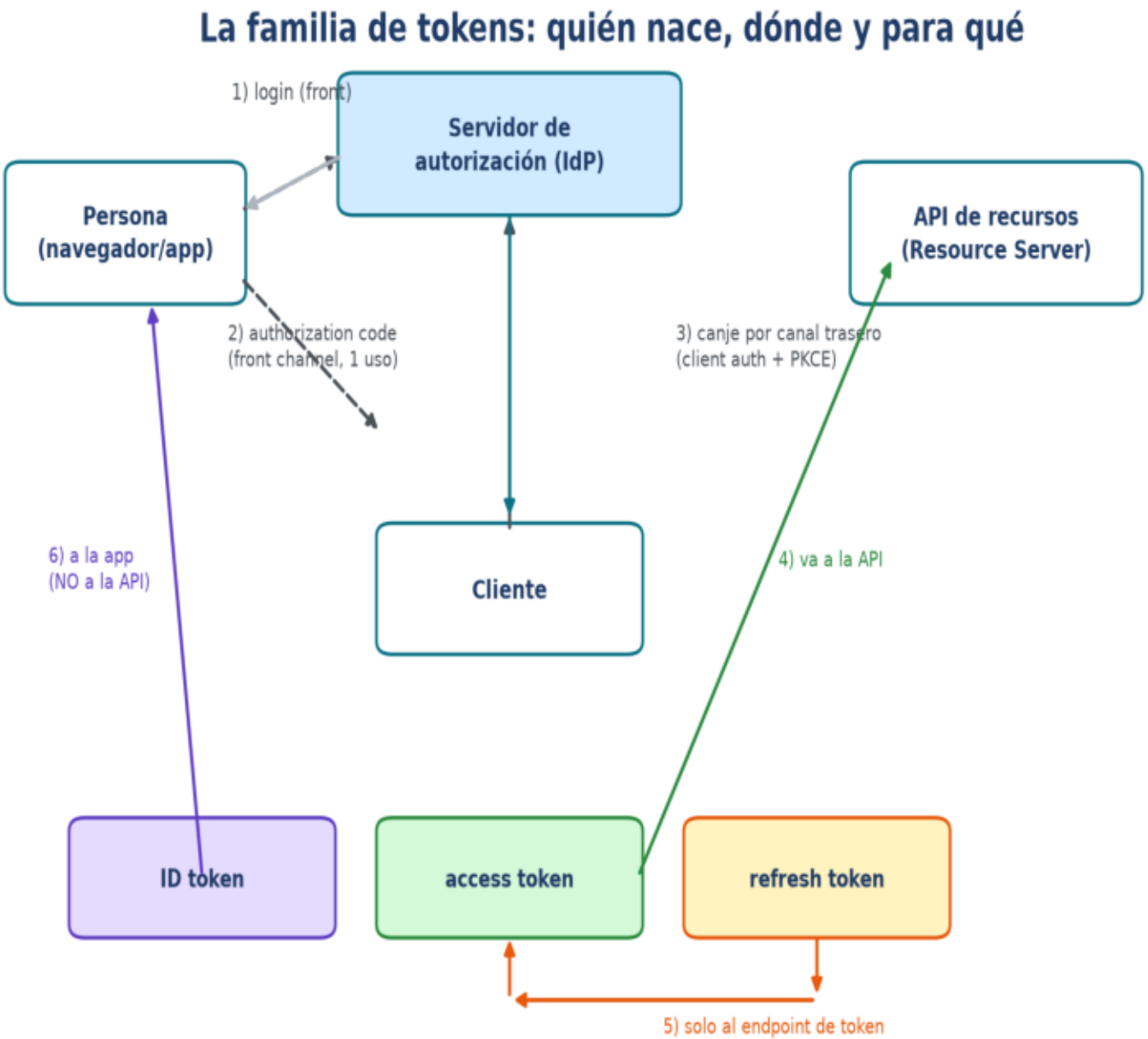
ANEXO — DIAGRAMAS

13 diagramas en página completa

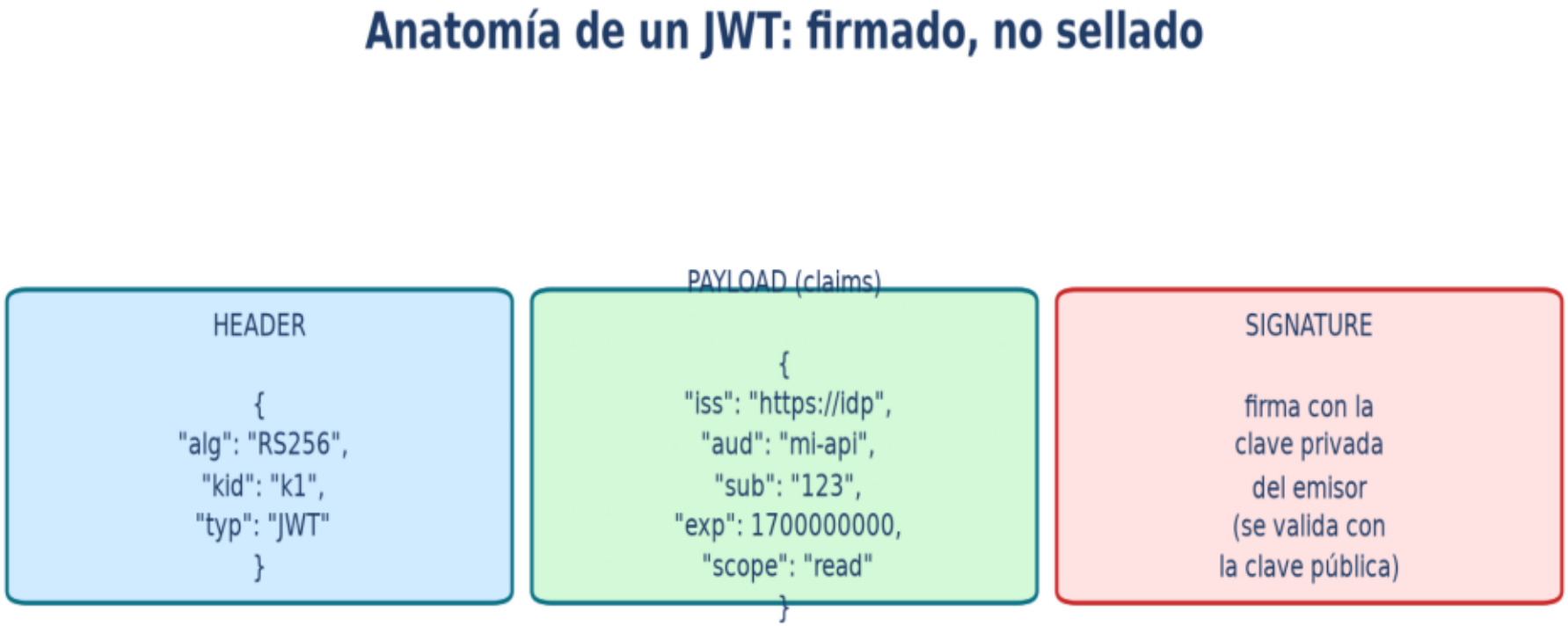
1. Línea de tiempo: de las credenciales al token atado



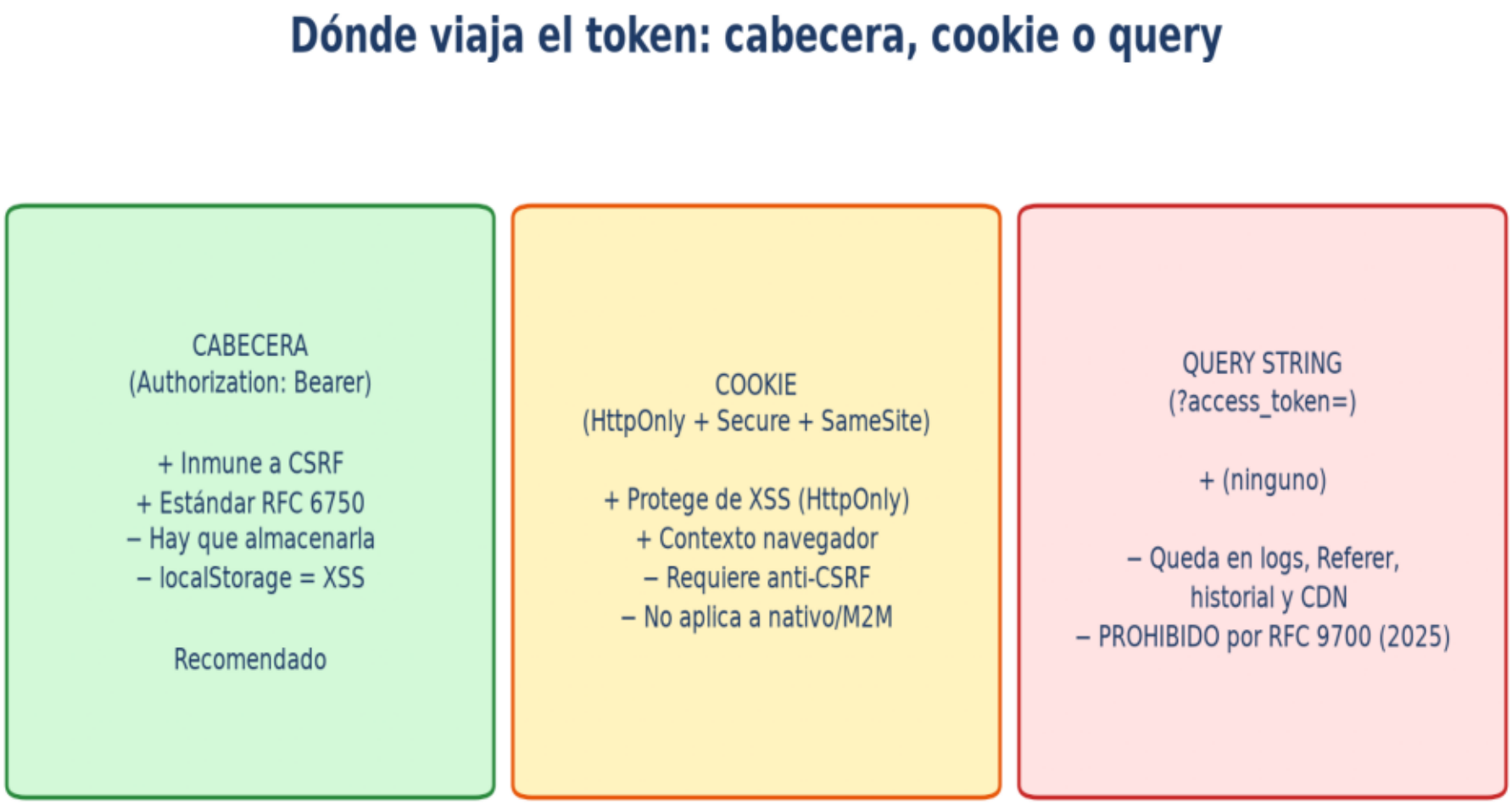
2. La familia de tokens y el flujo de emisión



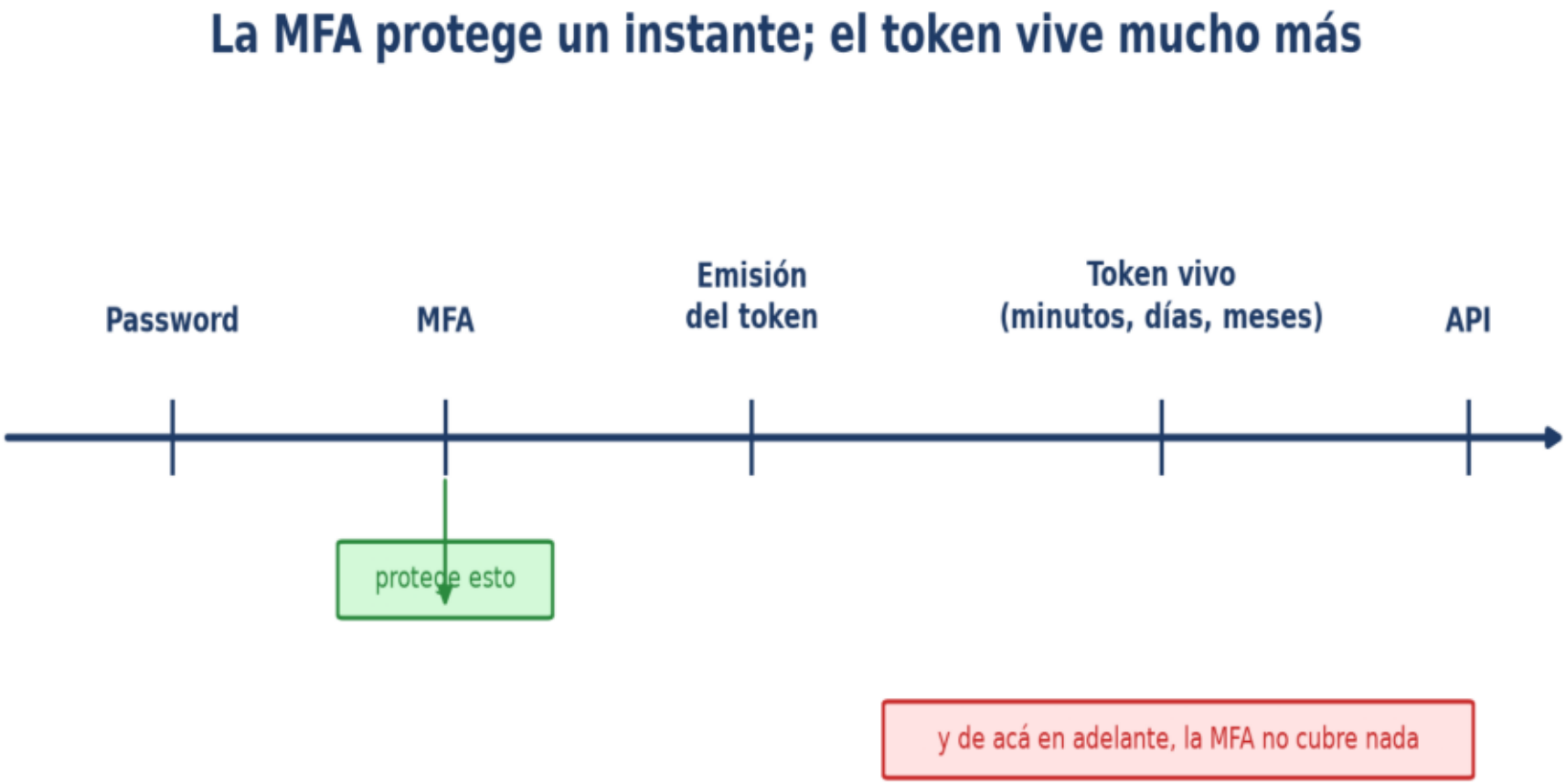
3. Anatomía de un JWT: firmado, no sellado



4. Dónde viaja el token: cabecera, cookie o query



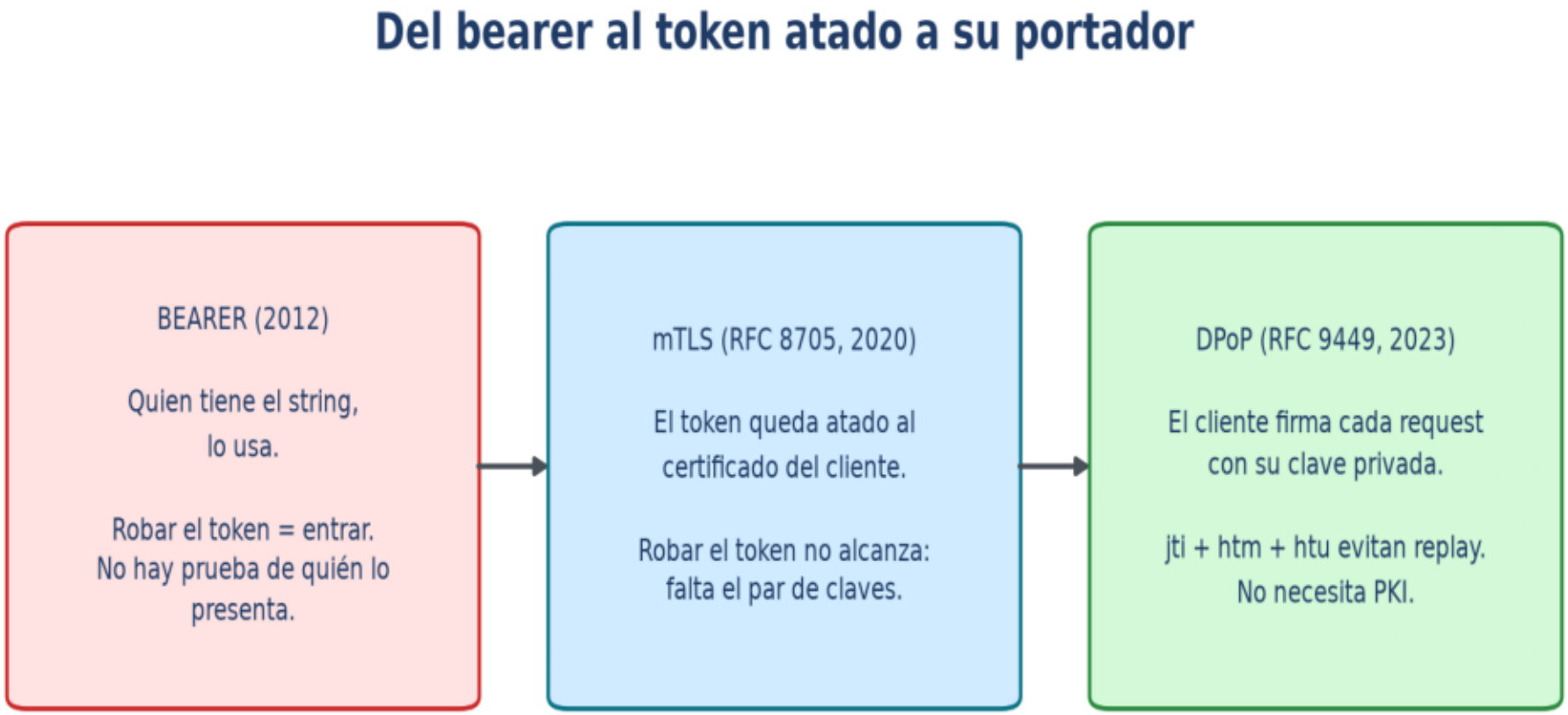
5. La MFA protege un instante; el token vive mucho más



6. Persona vs. sistema: dos preguntas distintas

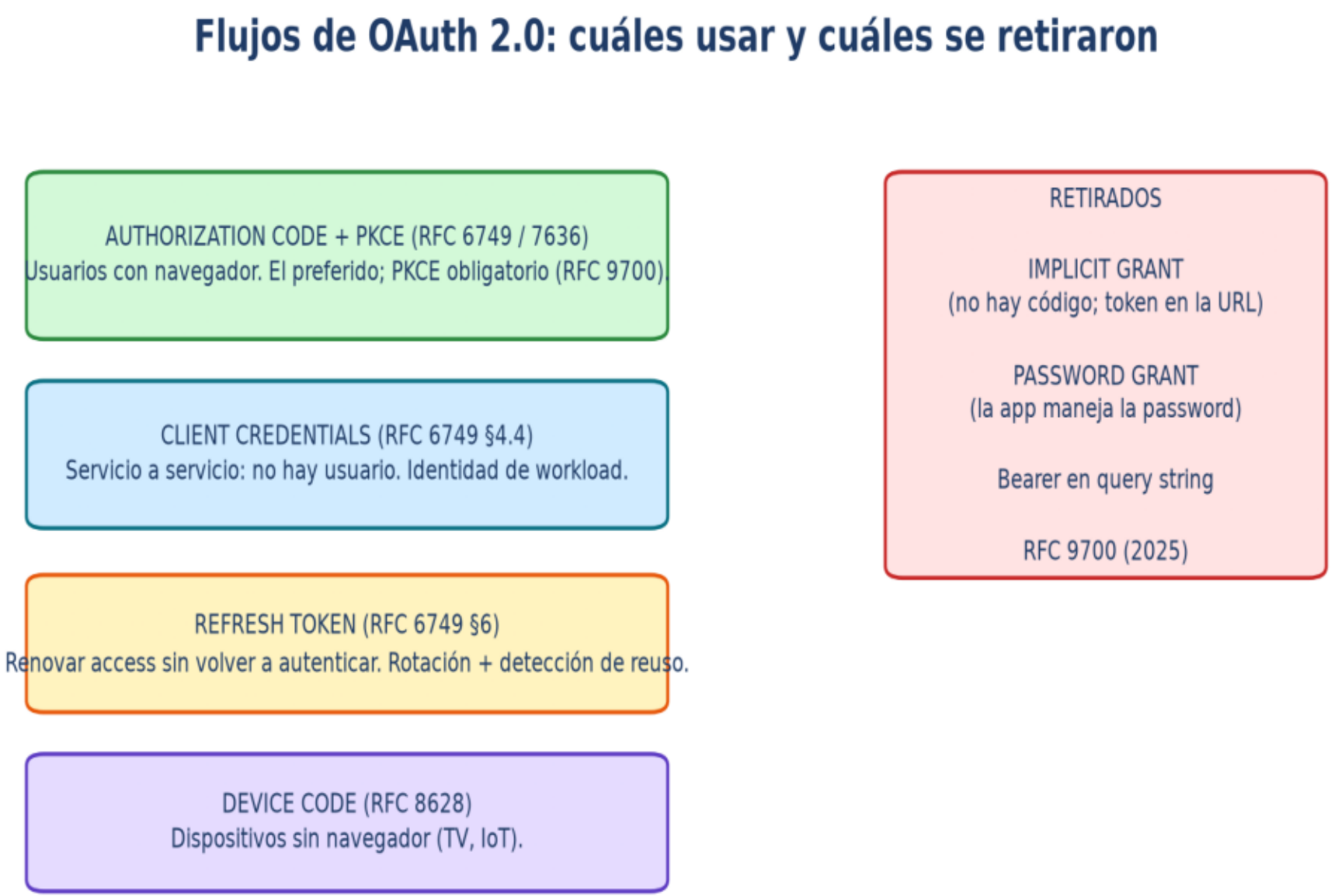


7. Del bearer a los tokens sender-constrained

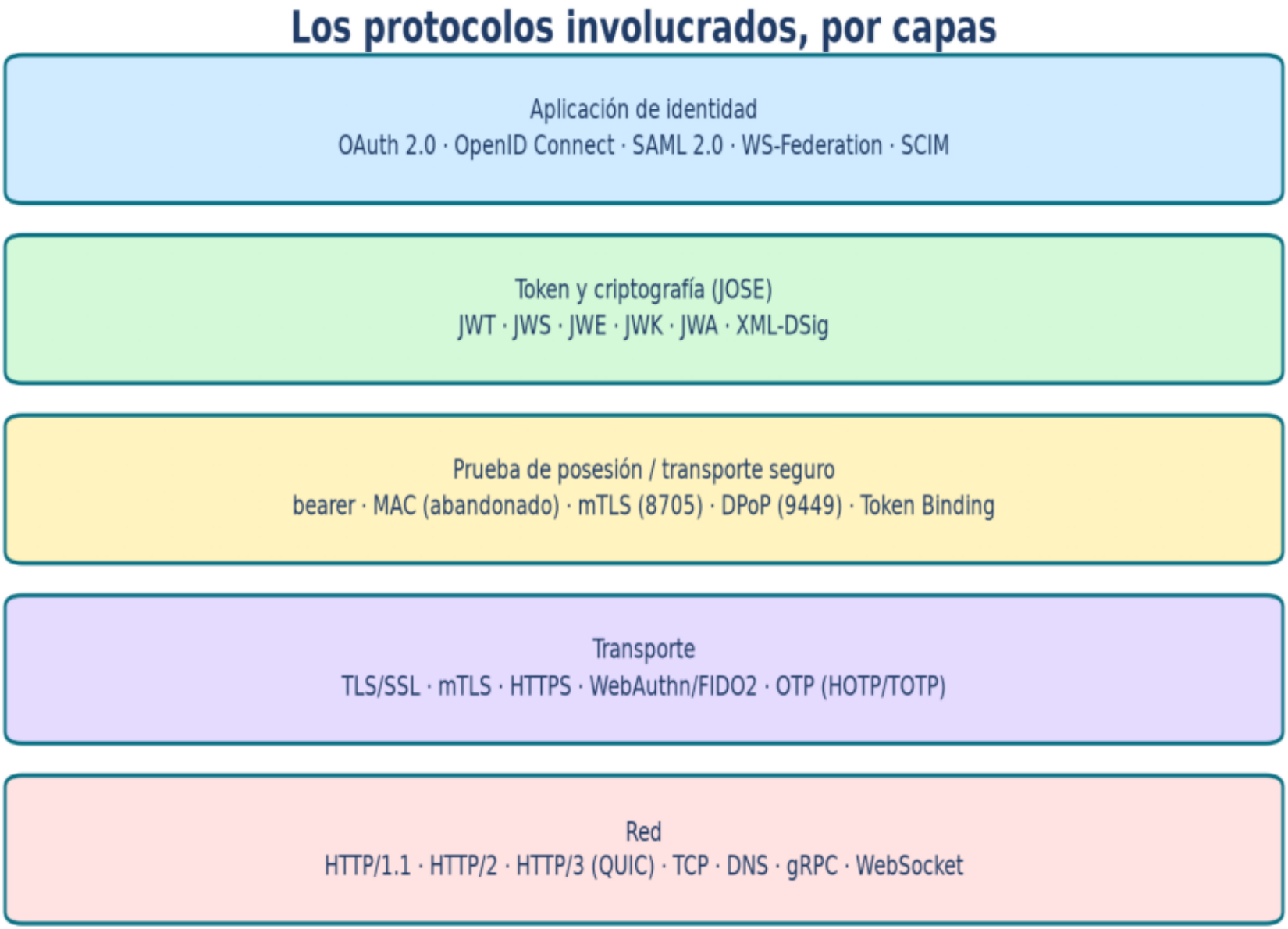


RFC 9700 (2025): los tokens sender-constrained se recomiendan donde sea posible.

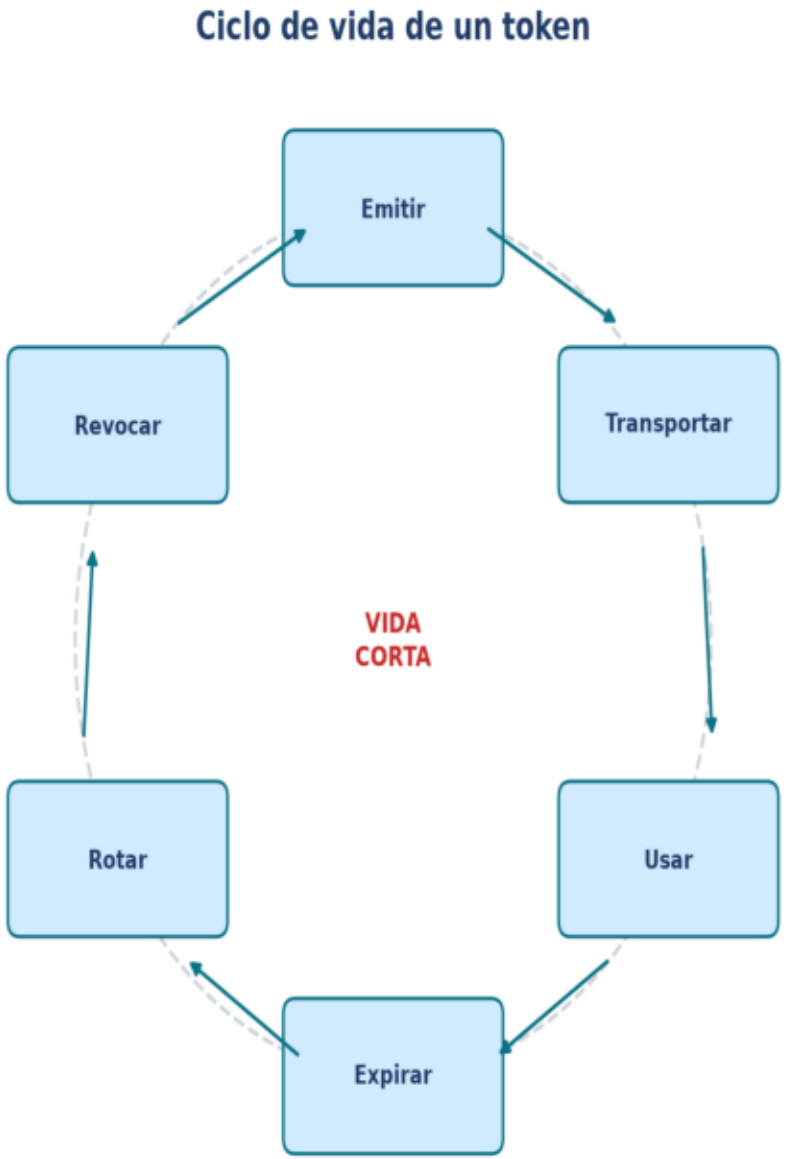
8. Flujos de OAuth 2.0: cuáles usar y cuáles se retiraron



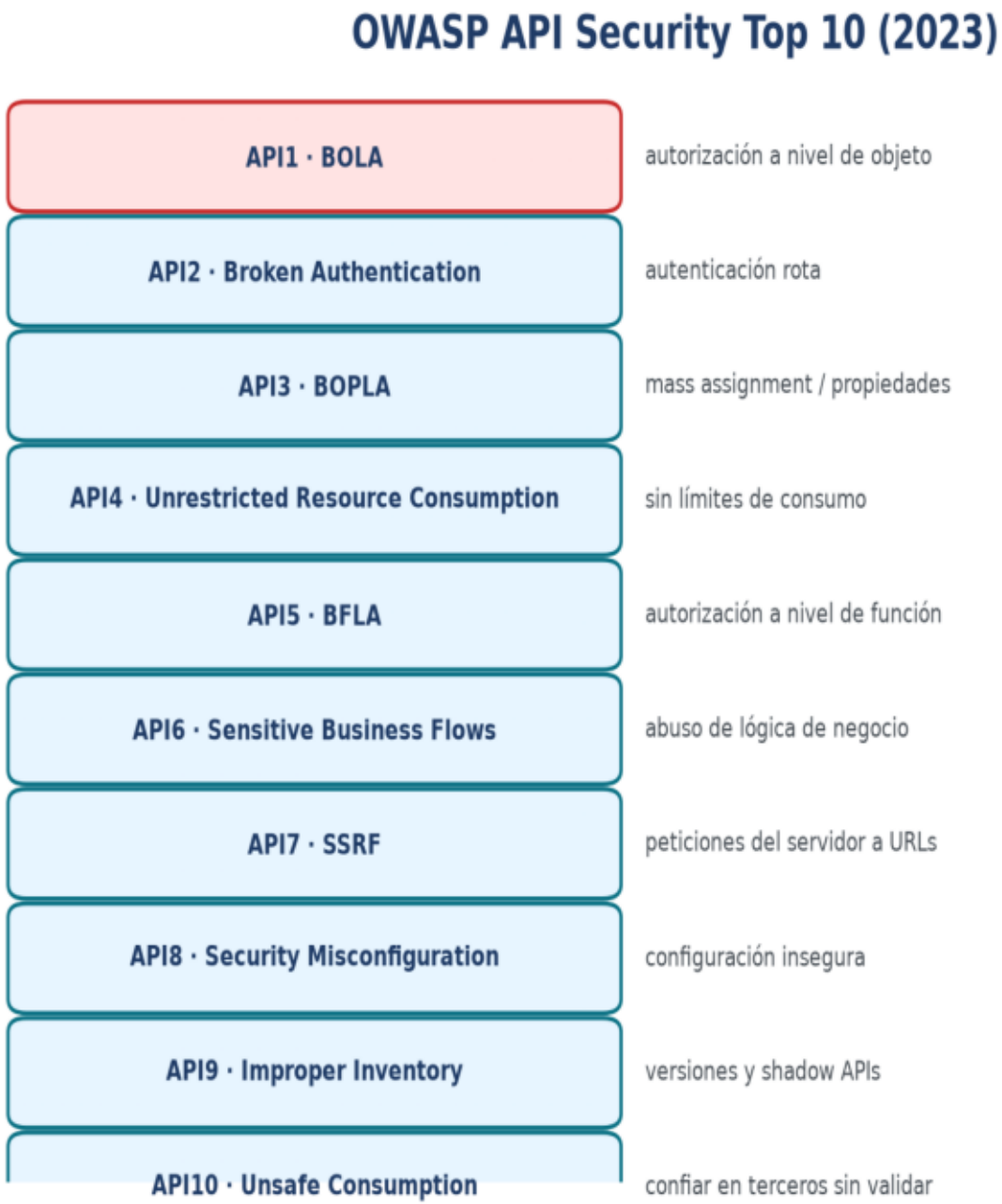
9. Los protocolos involucrados, por capas



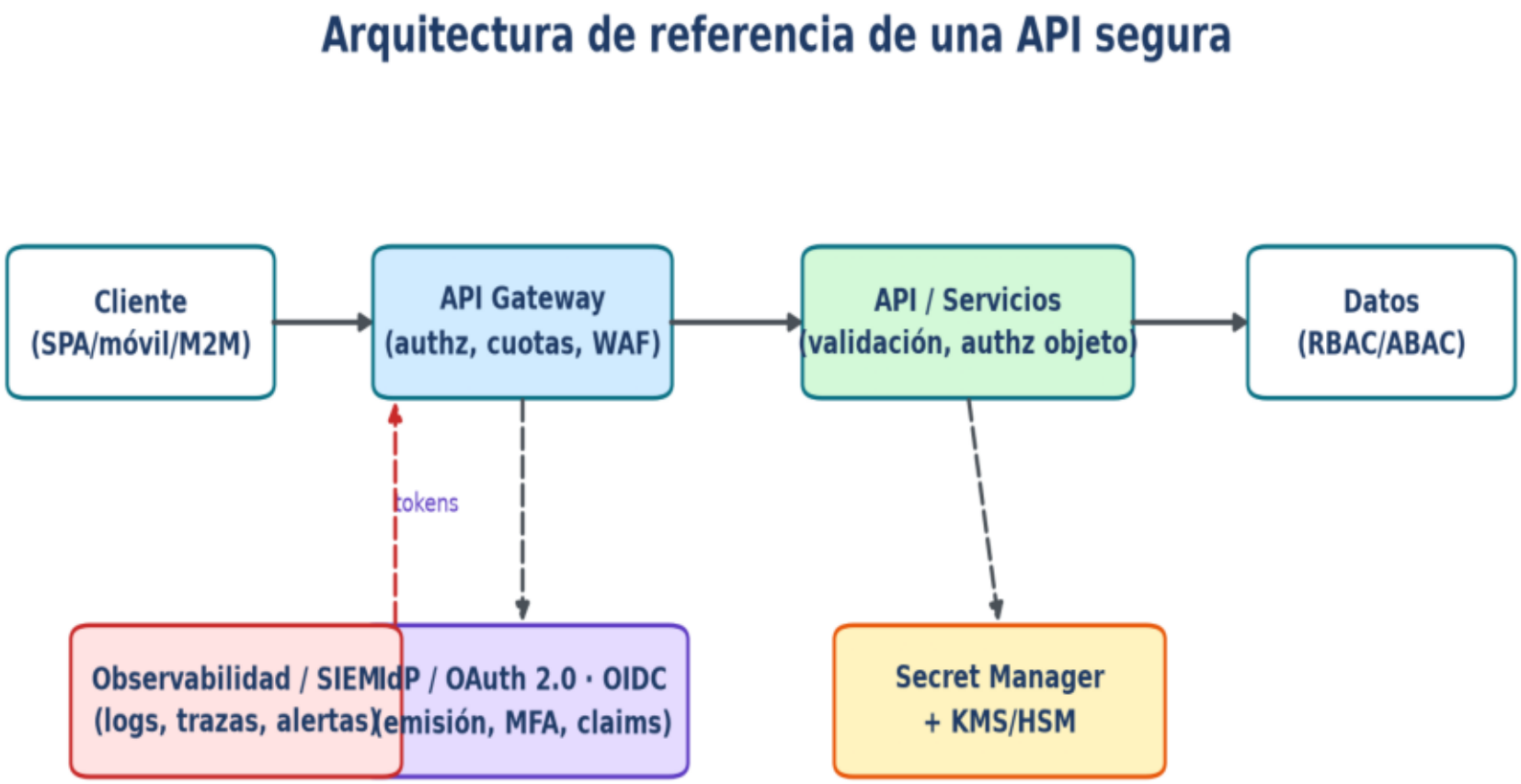
10. Ciclo de vida de un token



11. OWASP API Security Top 10 (2023)



12. Arquitectura de referencia de una API segura



Cada capa agrega un control. El token es solo una de ellas.

13. Alrededor del token hay un sistema entero

